

CSE 144 Final Project Report

Transfer Learning Challenge

Group Members (max 3):

Jake Davidson (CruzID: jsdavids)

Spring 2026

1 Introduction

1. Problem goal and setting.

The goal of the project is to classify a test set of around 1000 images across 100 categories using transfer learning from ImageNet-pretrained models, given a relatively small data set of around 1000 labeled training images.

2. Why transfer learning is appropriate.

Given the relatively small size of the labeled training dataset, transfer learning is appropriate to leverage the innate weights of pretrained models, while still allowing flexibility in fine-tuning.

3. Brief summary of your approach and main result.

To summarize my overall approach, I started by testing different ablations to augment the sparse labeled dataset on ResNet-152 and ViT-B/16 (some of the architectures we have discussed in the context of class lectures). Using some of the experiments as a baseline, I modified hyperparameters as necessary, implemented new data augmentation techniques, ensembled multiple backbone architectures, and used test time augmentation to produce my submissions.

The best ensemble submission had a public score on the Kaggle challenge of 0.8000, or 80%, and improved on the individual scores of the composite backbone architectures by as much as 9.1 percentage points.

2 Dataset

1. Number of classes and dataset sizes (train/val/test).

The labeled training dataset consists of 1079 images across 100 classes, of which I used 20% to form a validation set. While each class had around 10 images represented in the training set, the representation was imbalanced and ranged from as few as four images to as many as 41 images per class. Separate from the labeled training set, there was also a test set of 1036 images used to produce the submission.csv file. The test set of images was not labeled, for clear and obvious reasons.

2. Directory structure and label mapping details (ensure labels 0-99 match folders).

The main folder of the directory, named “Kaggle Transfer Learning”, contains the data from Kaggle in the folder named ucsc-cse-144-spring-2026-final-project (containing the same data in the same schema), as well as multiple python and jupyter notebooks representing the different network backbones, .pth files representing model checkpoints, .png files illustrating training/validation curves, .csv files containing results on the test set, and a requirements.txt file describing the requirements necessary for the virtual environment. One early challenge in the project was to ensure that labels were accurately mapped to folders, as integer labels would be sorted lexicographically (unintentionally mapping folder “10” to label 2 for example). To account for this, I overrode the dataset’s class-to-index map with a numeric one, such that folder “k” would be represented by label k.

3. Preprocessing (resize, normalization) and data augmentation used.

For preprocessing, the data is first split into the training and validation subset (80% and 20% respectively). The validation set is resized to 256x256, center cropped into a 224x224 image, and then converted into a tensor using ToTensor before being normalized with ImageNet normalization. Notably, no augmentation happens on the validation subset, resulting in a deterministic outcome. The training set is processed differently, applying augmentations like RandomRotation, RandomResizedCrop, RandomHorizontalFlip, and ColorJitter before being converted into a tensor and normalized with ImageNet normalization. Further data augmentation is applied during training, where MixUp and CutMix are applied randomly on a per batch basis.

3 Implementation

3.1 Model

1. Pretrained backbone used and why.

Initially, I used a jupyter notebook to fine-tune the ViT-B/16 and ResNet-152 backbone architectures (see vit.ipynb and resnet.ipynb). The sequential cells allowed me to visualize the fine-tuning process and accustomed me to the pipeline process. I chose these two models to start with as they were discussed in lecture. Using these notebooks as a launching pad, I also implemented the same pipeline with other models: EfficientNet B3, ConvNeXt small, Swin Transformer small, and ResNet-50. I relied on models that were described as computationally efficient while maintaining accuracy, as I planned on using ensemble methods and wanted to avoid long training times and tedious hyperparameter tuning. An intentional consideration was made to include Swin Transformer small to include additional transformers in the ensemble.

2. Architecture changes (classifier head, pooling, dropout, etc.)

Some key architectural changes included replacing the classifier heads of each model in the pipeline, replacing it with a 100-class output head. As mentioned earlier, per-batch augmentations were also included in the pipeline to artificially increase the variation in training data. The augmentations I chose to implement as training time were CutMix and MixUp, both of which used soft labels to inflate the training data available.

3. Fine-tuning strategy (frozen layers, unfreezing schedule).

My fine-tuning strategy revolved around replacing the classifier head while keeping the backbone parameters mostly the same with a lower learning rate (around 10% of the head learning rate in most cases, as described in lecture).

3.2 Training

1. Loss function and optimizer

I used soft cross-entropy (lam-weighted cross-entropy over y_a and y_b) in training to account for soft labels produced by MixUp and CutMix, and plain cross-entropy loss when calculating validation loss for real labels. For optimization I used AdamW with two learning rates, one for the head and one for the backbone.

2. Learning rate, scheduler, batch size, epochs, weight decay.

In general, I used a learning rate of 0.001 ($e-3$) for the head and 0.0001 ($e-4$) for the backbone. This followed the 10% learning rate “rule of thumb” discussed in lecture while allowing both the head and backbone to adjust to the training data. For scheduling, I used Cosine Annealing learning rate to allow for some ability to overcome local minima while still scaling the learning rate appropriately to the time passed. In general, I used a batch size of 12 to account for limited hardware, though I had to further limit the batch size to 8 when fine-tuning

EfficientNet B3 due to out of memory errors. I also standardized the number of epochs to 30 to get a consistent visualization in the training/validation curves for analysis. The weight decay for AdamW was set to 0.0001 ($e-4$).

3. Hardware/software environment (GPU, PyTorch version)

I trained the models locally on my RTX 960 4GB GPU, hence the aforementioned limited hardware, and used torch version 2.12.0 and torchvision version 0.27.0. I used Windows 10 and Python 3.14.

4 Experiments

1. Baseline setup.

For my initial experiments, I tested the validation performance using a baseline setup of ResNet-152 across 30 epochs, with batch size 12 across a variety of variables. I tried freezing the entire backbone, lowering learning rates for the backbone, introducing more and stronger data augmentation techniques. After comparing the training and validation curves for these experiments I also tried submitting the individual models and the ensemble model to Kaggle with and without test time augmentation to verify the effectiveness of such a technique.

2. Hyperparameter tuning plan and validation method.

I planned a lot of my experiments around verifying improvements in validation accuracy when tuning hyperparameters, but also made sure to observe when the training accuracy crossed the thresholds of 90% and 95% to check for model overfitting. Another detail I kept in mind was the 60% baseline score required to pass the final project. I did so by drawing a 60% line on the validation curve to visualize the performance of a model post-experiment in the context of submitting the results to Kaggle.

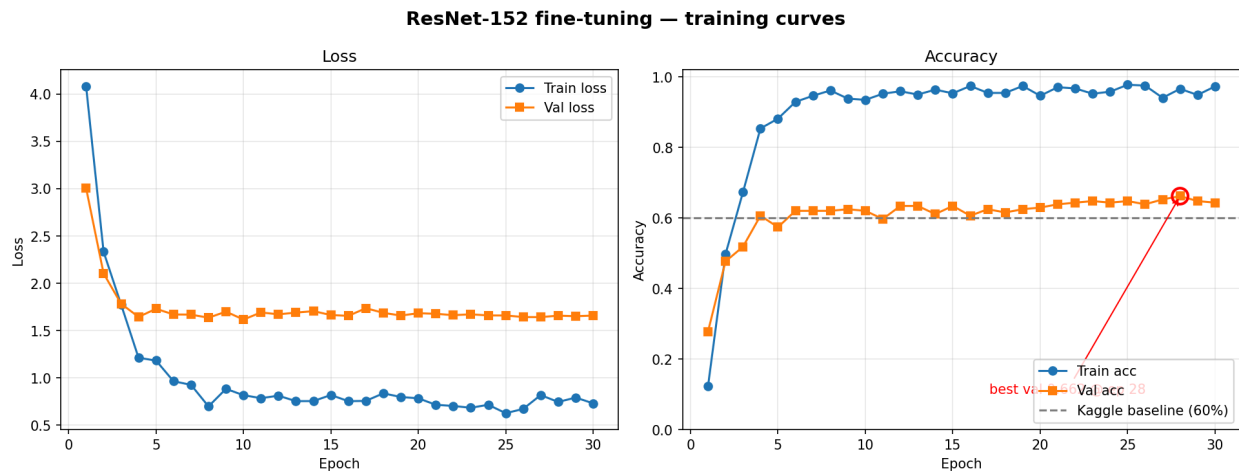
3. Ablations (augmentations, model size, freezing strategy, LR, etc.).

I started with per image augmentations, including rotations, flipping, and color jitter. Then, I implemented MixUp and CutMix to introduce artificial data into the training process. Additionally, I implemented test time augmentation in an attempt to improve the robustness of the ensembled models. I also tried a variety of model sizes after starting with ResNet-152 and ViT-B/16, trying ResNet-50, various ConvNeXt models, various EfficientNet models, and various Swin Transformer models. Some additional considerations were made to lower the learning rate for the Swin Transformer and Vision Transformer as the PyTorch demos had explained that the transformer models would be more sensitive to the learning rate.

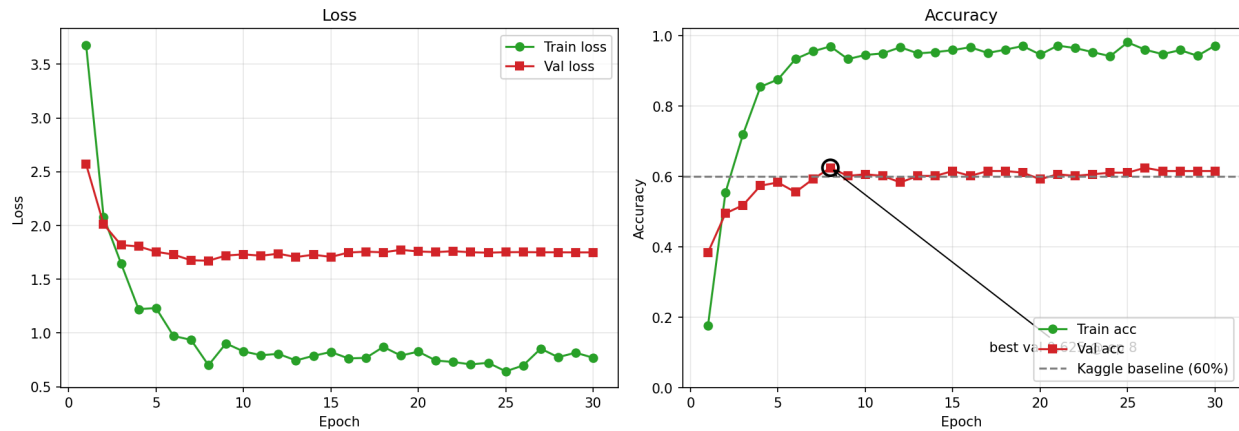
5 Results

1. Training/validation accuracy (and loss) and any key curves/plots.

The following are the initial loss and accuracy curves for ResNet-152 and ViT-B/16.

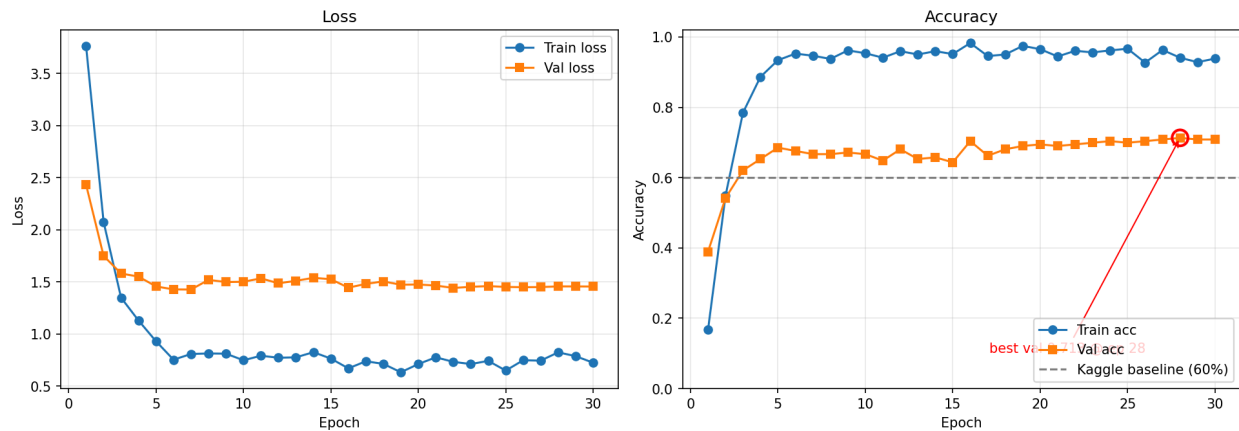


ViT-B/16 fine-tuning — training curves

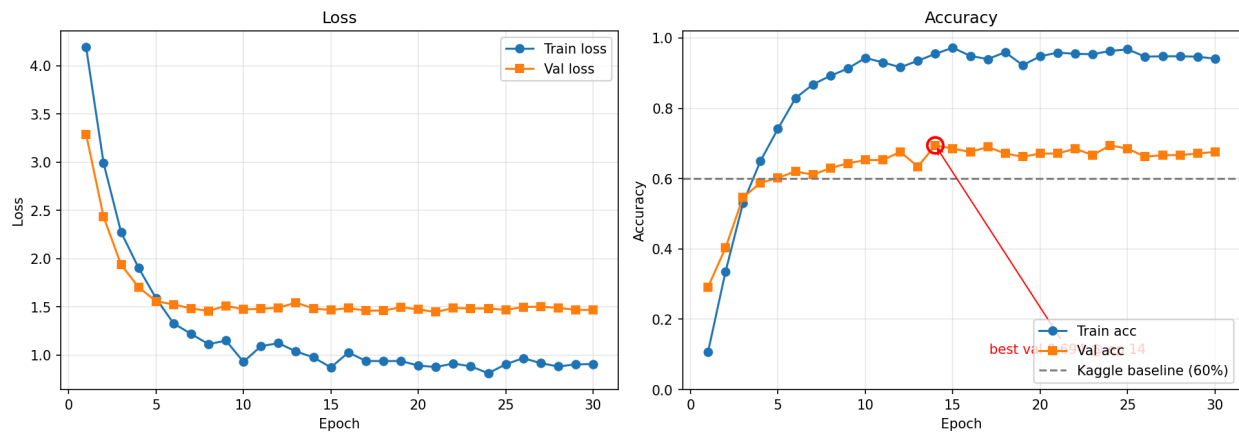


Next, I tried ensembling a few different models, with the following loss and accuracy train/val curves:

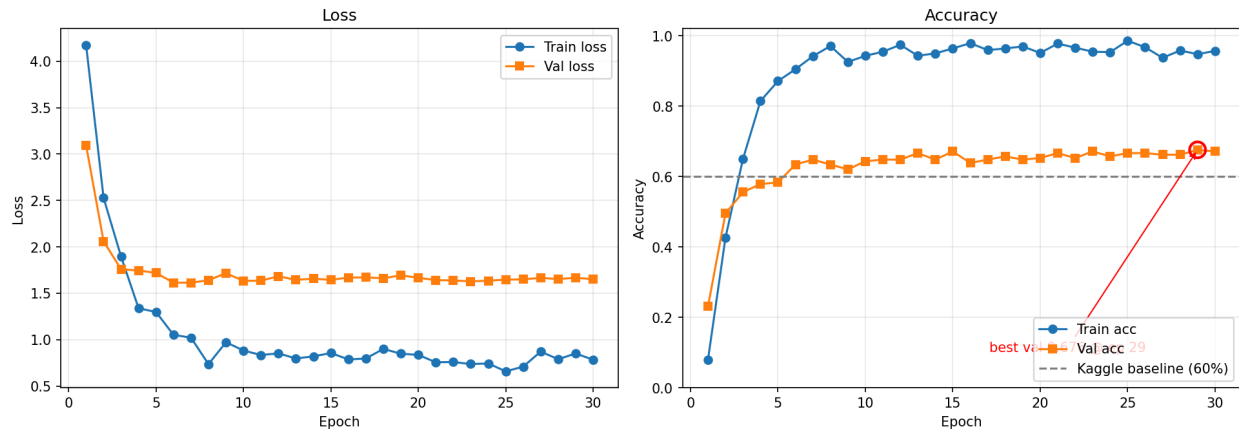
convnext_small fine-tuning — training curves



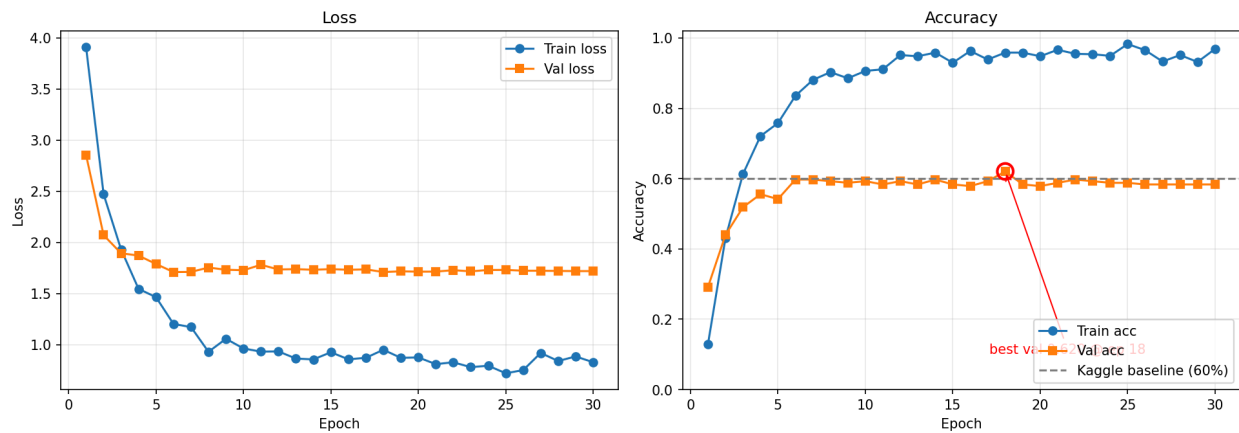
efficientnet_b3 fine-tuning — training curves



resnet50 fine-tuning — training curves



swin_s fine-tuning — training curves



2. Kaggle public leaderboard score (and submission settings).

Below are some key public leaderboard scores and some notes on the models/output submitted.

Submission Title	Model/Notes	Leaderboard Score (Public)
submission_resnet.csv	ResNet-152 notebook	0.75454
submission_vit.csv	ViT-B/16 notebook	0.76363
submission_convnext_small.csv	convnext_small via python command	0.75454
submission_efficientnet_b3.csv	efficientnet_b3 via python command	0.73636
submission_resnet50.csv	ResNet-50 via python command	0.75454

submission_swin_s.csv	swin_s via python command	0.70909
submission_ensemble.csv	ensemble of convnext, efficientnet, swin, and vit	0.76363
submission_ensemble_tta.csv	ensemble of convnext, efficientnet, swin, and vit using test time augmentation	0.80000

6 Discussion

1. What worked best and why.

I found that using ensemble methods over ConvNeXt small, EfficientNet B3, Swin Transformer small, and ViT-B/16 with test time augmentation worked the best for me. I believe this is a result of using multiple types of architectures (both convolution networks and vision transformers) and introducing many types of data augmentation (per image rotating/flipping/color mixing, per batch CutMix/MixUp, and test time augmentation) to artificially inflate the available data and robustness of the network.

2. Failure cases, overfitting/underfitting observations.

I often found that per-model performance often peaked before epoch 10, and often showed signs of overfitting with training accuracy pushing 95%+ while validation accuracy stagnated around 60%. These observations are what drove me to more and more augmentation techniques, pushing me from image augmentation to batch augmentation and finally to test time augmentation.

3. Limitations and concrete next improvements.

Some clear limitations I saw included overfitting. Some concrete next improvements might include using stronger augmentation techniques, like PyTorch's RandAugment, as well as

studying the effects of gradual unfreezing to gradually close the gap between training and validation accuracy. Another area of improvement might include using K-fold validation to ensure access to the entirety of the labeled data set during training, while also reducing overfitting in training.

7 Reproducibility

1. Random seeds and determinism settings.

Although I eventually added additional determinism settings to set the seed, for the experiments and evaluation listed here I used a random seed of 42.

2. Package versions / environment setup steps.

The torch and torchvision versions I used:

```
torch==2.12.0
```

```
torchvision==0.27.0
```

```
numpy==2.4.2
```

```
matplotlib==3.10.8
```

```
pillow==12.1.1
```

```
jupyter
```

Any recent version of jupyter should work, I just installed the most recent version. To set up the entire environment, I used a python virtual environment, then installed the requirements using pip as follows:

```
pip install torch==2.12.0 torchvision==0.27.0 numpy==2.4.2  
matplotlib==3.10.8 pillow==12.1.1 jupyter
```

3. Exact commands to train and to generate `submission.csv`.

The initial jupyter notebooks, `resnet.ipynb` and `vit.ipynb`, generated `submission.csv` and `training_curve.png` as a result of running each cell in the notebook sequentially. For the ensemble output, the required commands to produce the output are:

```
python run_model.py convnext_s
python run_model.py efficientnet_b3 --batch-size 8
python run_model.py swin_s
python run_model.py vit_b16
python ensemble.py convnext_s efficientnet_b3 swin_s
vit_b16 --tta
```

This sequence of commands produces `training_curve.png` and `submission.csv` files for each individual backbone architecture, as well as a `submission_ensemble.csv` file ensembling the four architectures and implementing test time augmentation for a more robust output.

8 Team Contributions

1. Member 1 (Jake Davidson):

I pursued the Kaggle transfer learning project as a solo-group, so the entirety of the work, in model research, training, evaluation, and reporting, is my own.

9 References

1. TorchVision documentation / pretrained model sources.

Pretrained models <https://docs.pytorch.org/vision/main/models.html>:

ConvNeXt (`convnext_small`):

<https://docs.pytorch.org/vision/main/models/convnext.html>

EfficientNet (efficientnet_b3):

<https://docs.pytorch.org/vision/main/models/efficientnet.html>

ResNet (resnet50 and resnet152):

<https://docs.pytorch.org/vision/main/models/resnet.html>

SwinTransformer (swin_s):

https://docs.pytorch.org/vision/main/models/swin_transformer.html

VisionTransformer (vit_b_16):

https://docs.pytorch.org/vision/main/models/vision_transformer.html

2. Any papers or tutorials used.

TorchVision Documentation to reference methods and attributes:

<https://docs.pytorch.org/vision/main/models.html>

PyTorch's Transfer Learning for Computer Vision Tutorial:

https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html